CO2 – Lab Experiments

NAME: THAMIZHARASAN S

REG NO: 192424087

COURSE CODE: CSA4708

COURSE NAME: DEEP LEARNING

1. Perceptron and Forward Propagation

Question 1: Smart Traffic Signal Classification

A smart traffic system classifies traffic conditions as Low Traffic (0) or Heavy Traffic (1) using vehicle count and average waiting time. A perceptron produces incorrect predictions for some input combinations because the weights and bias are not properly adjusted.

Implement a perceptron using NumPy to calculate the weighted sum and apply a step activation function. Use different weight and bias values to analyze their effect on classification accuracy.

Plot the input data and decision boundary using Matplotlib, and identify which parameter changes improve the classification results.

Packages: NumPy, Matplotlib

AIM:

To implement a perceptron using NumPy to classify traffic conditions as Low Traffic (0) or Heavy Traffic (1) based on vehicle count and average waiting time, and analyze the effect of weights and bias on classification accuracy using Matplotlib.

PROCEDURE:

- Import NumPy and Matplotlib libraries.
- Create input data containing vehicle count and average waiting time.
- Assign target labels: Low Traffic (0) and Heavy Traffic (1).
- Calculate the weighted sum using weights and bias.
- Apply the step activation function to predict traffic conditions.
- Change weights and bias to compare classification accuracy.
- Plot the input data and decision boundary using Matplotlib.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

X = np.array([

    [10, 5],

    [20, 10],

    [30, 15],

    [60, 30],

    [80, 40],

    [100, 50]

])

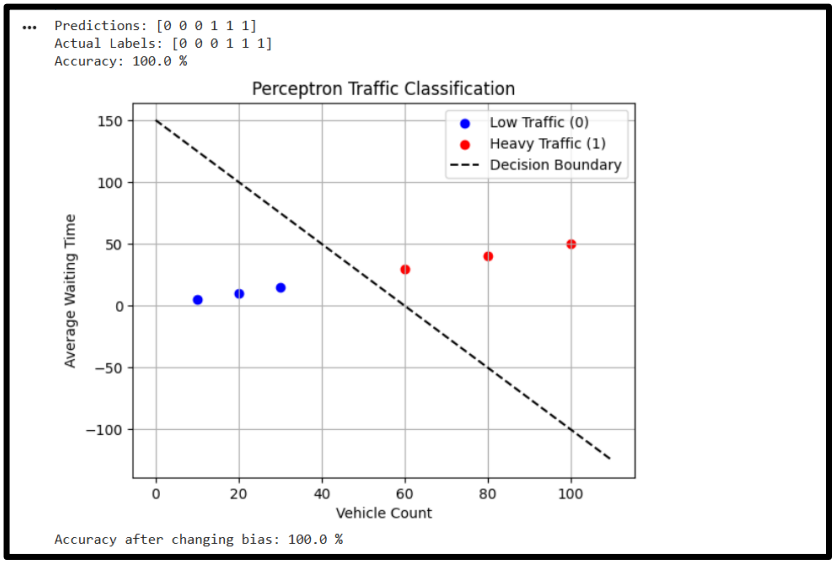
y = np.array([0, 0, 0, 1, 1, 1])

weights = np.array([0.05, 0.02])
```

```
bias = -3.0
z = np.dot(X, weights) + bias
predictions = (z >= 0).astype(int)
accuracy = np.mean(predictions == y) * 100
print("Predictions:", predictions)
print("Actual Labels:", y)
print("Accuracy:", accuracy, "%")
plt.scatter(X[y == 0, 0], X[y == 0, 1],
            color='blue', label='Low Traffic (0)')
plt.scatter(X[y == 1, 0], X[y == 1, 1],
            color='red', label='Heavy Traffic (1)')
x_line = np.linspace(0, 110, 100)
y_line = -(weights[0] * x_line + bias) / weights[1]
plt.plot(x_line, y_line, 'k--', label='Decision Boundary')
plt.xlabel("Vehicle Count")
plt.ylabel("Average Waiting Time")
plt.title("Perceptron Traffic Classification")
plt.legend()
plt.grid(True)
plt.show()
weights = np.array([0.05, 0.02])
bias = -2.0
predictions = (np.dot(X, weights) + bias >= 0).astype(int)
```

```
accuracy = np.mean(predictions == y) * 100
print("Accuracy after changing bias:", accuracy, "%")
```

INPUT & OUTPUT:



RESULT:

The perceptron successfully classified traffic conditions using NumPy. The Matplotlib graph visualized the decision boundary, and parameter changes demonstrated their effect on classification accuracy.

Question 2: Industrial Machine Status Prediction

An industrial monitoring system predicts whether a machine requires inspection (1) or can continue operating (0) using temperature and vibration measurements. During testing, some machines are incorrectly classified because the weighted inputs do not produce the expected output.

Implement a perceptron and perform forward propagation for the given input samples. Calculate the weighted sum, apply the activation function, and compare the predicted outputs with the actual labels. Modify the weights and bias, visualize the classification results, and analyze the limitations of the perceptron for the given dataset.

Packages: NumPy, Matplotlib

AIM:

To implement a perceptron using NumPy to predict industrial machine status based on temperature and vibration, calculate weighted sums, apply a step activation function, and analyze classification accuracy using Matplotlib.

PROCEDURE:

- Import NumPy and Matplotlib libraries.
- Create input samples containing temperature and vibration measurements.
- Assign actual labels: 0 for Continue Operating and 1 for Requires Inspection.
- Initialize weights and bias.
- Calculate the weighted sum using \(z=w_1x_1+w_2x_2+b\).
- Apply the step activation function to predict machine status.
- Compare predicted outputs with actual labels and calculate accuracy.

- Modify weights and bias, then visualize the classification results and decision boundary.
- Analyze the limitations of the perceptron.

PROGRAM CODE:

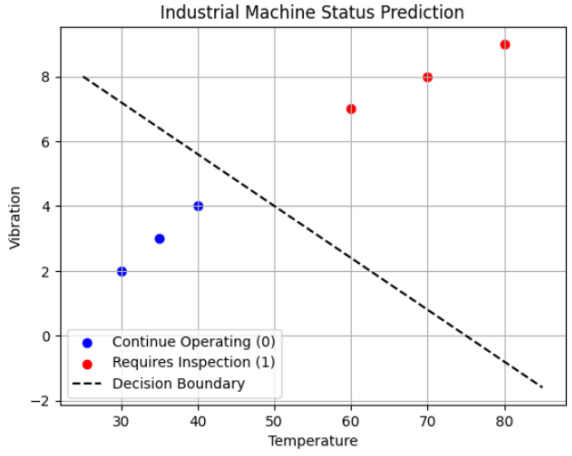
```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([
    [30, 2],
    [35, 3],
    [40, 4],
    [60, 7],
    [70, 8],
    [80, 9]
])
y = np.array([0, 0, 0, 1, 1, 1])
weights = np.array([0.05, 0.5])
bias = -5.0
z = np.dot(X, weights) + bias
predictions = (z >= 0).astype(int)
print("Weighted Sums:", z)
print("Actual Labels:", y)
print("Predicted Labels:", predictions)
accuracy = np.mean(predictions == y) * 100
```

```
print("Initial Accuracy:", accuracy, "%")
weights = np.array([0.08, 0.5])
bias = -6.0
z = np.dot(X, weights) + bias
predictions = (z >= 0).astype(int)
accuracy = np.mean(predictions == y) * 100
print("\nModified Predictions:", predictions)
print("Modified Accuracy:", accuracy, "%")
plt.scatter(X[y == 0, 0], X[y == 0, 1],
            color='blue', label='Continue Operating (0)')
plt.scatter(X[y == 1, 0], X[y == 1, 1],
            color='red', label='Requires Inspection (1)')
x_line = np.linspace(25, 85, 100)
y_line = -(weights[0] * x_line + bias) / weights[1]
plt.plot(x_line, y_line, 'k--', label='Decision Boundary')
plt.xlabel("Temperature")
plt.ylabel("Vibration")
plt.title("Industrial Machine Status Prediction")
plt.legend()
plt.grid(True)
plt.show()
```

INPUT & OUTPUT:

```
Weighted Sums: [-2.5  -1.75 -1.    1.5   2.5   3.5 ]
Actual Labels: [0 0 0 1 1 1]
Predicted Labels: [0 0 0 1 1 1]
Initial Accuracy: 100.0 %

Modified Predictions: [0 0 0 1 1 1]
Modified Accuracy: 100.0 %
```



RESULT:

The perceptron successfully performed forward propagation and classified machine status. The visualization showed the decision boundary, while the parameter changes demonstrated how weights and bias influence predictions.

2. Two-Layer Neural Network with Backpropagation

Question 3: Electricity Consumption Prediction

A smart building uses temperature and the number of occupied rooms to predict electricity consumption. A two-layer neural network initially produces large prediction errors because its weights are randomly initialized.

Implement a two-layer neural network using NumPy, with a hidden layer and an output layer. Perform forward propagation to calculate the predicted consumption, compute the loss using mean squared error, and apply backpropagation to update the weights and biases. Store the loss values and plot the loss curve using Matplotlib. Analyze how the weight updates influence prediction error over successive epochs.

Packages: NumPy, Matplotlib

AIM:

To implement a two-layer neural network using NumPy to predict electricity consumption based on temperature and occupied rooms, apply backpropagation to update weights and biases, and visualize the loss curve using Matplotlib.

PROCEDURE:

- Import NumPy and Matplotlib libraries.
- Create input data containing temperature and occupied rooms.
- Initialize weights and biases randomly.
- Perform forward propagation through the hidden and output layers.
- Calculate prediction error using Mean Squared Error (MSE).
- Apply backpropagation to calculate gradients and update weights and biases.

- Store loss values for successive epochs.
- Plot the loss curve and analyze the reduction in prediction error.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

X = np.array([

    [20, 2],

    [25, 3],

    [30, 4],

    [35, 5],

    [40, 6]

], dtype=float)

y = np.array([[30], [40], [50], [60], [70]], dtype=float)

X = (X - X.mean(axis=0)) / X.std(axis=0)

y_mean, y_std = y.mean(), y.std()

Y = (y - y_mean) / y_std

W1 = np.random.randn(2, 4) * 0.1

b1 = np.zeros((1, 4))

W2 = np.random.randn(4, 1) * 0.1

b2 = np.zeros((1, 1))

lr = 0.01
```

```
epochs = 1000
losses = []
for epoch in range(epochs):
    H = np.tanh(X @ W1 + b1)
    output = H @ W2 + b2
    error = output - Y
    loss = np.mean(error ** 2)
    losses.append(loss)
    d_output = 2 * error / len(X)
    dW2 = H.T @ d_output
    db2 = np.sum(d_output, axis=0, keepdims=True)
    d_hidden = (d_output @ W2.T) * (1 - H ** 2)
    dW1 = X.T @ d_hidden
    db1 = np.sum(d_hidden, axis=0, keepdims=True)
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1
    b1 -= lr * db1
H = np.tanh(X @ W1 + b1)
predicted = (H @ W2 + b2) * y_std + y_mean
print("Actual Consumption:", y.flatten())
print("Predicted Consumption:", np.round(predicted.flatten(),
2))
print("Initial Loss:", round(losses[0], 4))
```

```
print("Final Loss:", round(losses[-1], 4))

plt.plot(losses)

plt.xlabel("Epoch")

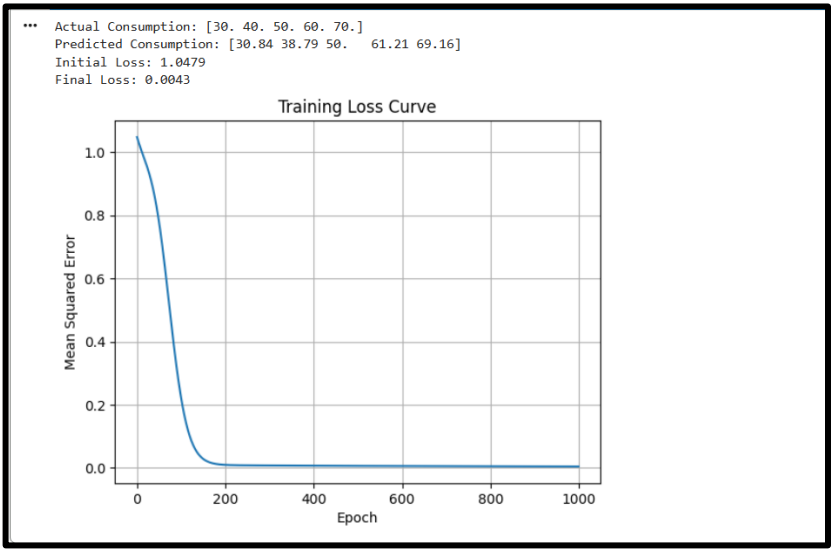
plt.ylabel("Mean Squared Error")

plt.title("Training Loss Curve")

plt.grid(True)

plt.show()
```

INPUT & OUTPUT:



RESULT:

The two-layer neural network was implemented successfully. Backpropagation updated weights and biases, reduced prediction error, and the loss curve visualized training progress.

Question 4: Crop Yield Classification

An agricultural monitoring system predicts whether crop yield will be high or low based on soil moisture and fertilizer quantity. The initial neural network misclassifies several samples, even though the input features contain useful information.

Implement a two-layer neural network using NumPy. Calculate the forward-pass outputs, binary classification loss, gradients, and updated weights using backpropagation. Compare the model's predictions before and after training. Plot the loss against the number of epochs and analyze whether the network learns an effective relationship between the input features and crop yield.

Packages: NumPy, Matplotlib

AIM:

To implement a two-layer neural network using NumPy to classify crop yield as High (1) or Low (0) based on soil moisture and fertilizer quantity, apply backpropagation to update weights, and visualize the training loss using Matplotlib.

PROCEDURE:

- Import NumPy and Matplotlib libraries.
- Create input samples for soil moisture and fertilizer quantity.
- Assign target labels: High Yield (1) and Low Yield (0).
- Initialize weights and biases randomly.
- Perform forward propagation using a hidden layer and sigmoid output layer.
- Calculate binary cross-entropy loss.

- Compute gradients and update weights and biases using backpropagation.
- Compare predictions before and after training.
- Plot the loss curve against the number of epochs and analyze the results.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

X = np.array([

    [20, 10],

    [25, 15],

    [30, 20],

    [60, 50],

    [70, 60],

    [80, 70]

], dtype=float)

y = np.array([[0], [0], [0], [1], [1], [1]])

X = (X - X.mean(axis=0)) / X.std(axis=0)

W1 = np.random.randn(2, 4) * 0.1

b1 = np.zeros((1, 4))

W2 = np.random.randn(4, 1) * 0.1

b2 = np.zeros((1, 1))

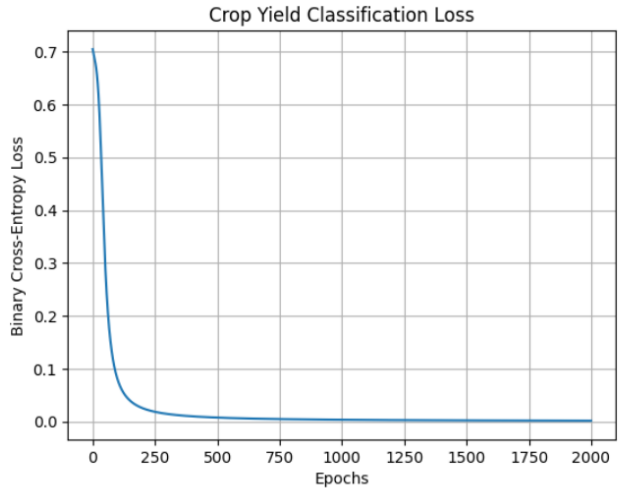
def sigmoid(z):
```

```
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
def forward(X):
    hidden = np.tanh(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return hidden, output
def binary_loss(pred, target):
    pred = np.clip(pred, 1e-7, 1 - 1e-7)
    return -np.mean(
        target * np.log(pred) +
        (1 - target) * np.log(1 - pred)
    )
_, before = forward(X)
print("Before Training:", (before >= 0.5).astype(int).flatten())
lr = 0.1
epochs = 2000
losses = []
for epoch in range(epochs):
    hidden, output = forward(X)
    loss = binary_loss(output, y)
    losses.append(loss)
    dZ2 = (output - y) / len(X)
    dW2 = hidden.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)
```

```
    dH = dZ2 @ W2.T
    dZ1 = dH * (1 - hidden ** 2)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1
    b1 -= lr * db1
_, after = forward(X)
predictions = (after >= 0.5).astype(int)
print("Actual Labels:", y.flatten())
print("After Training:", predictions.flatten())
print("Final Loss:", round(losses[-1], 4))
print("Accuracy:", np.mean(predictions == y) * 100, "%")
plt.plot(losses)
plt.xlabel("Epochs")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Crop Yield Classification Loss")
plt.grid(True)
plt.show()
```

INPUT & OUTPUT:

```
Before Training: [1 1 1 0 0 0]
Actual Labels: [0 0 0 1 1 1]
After Training: [0 0 0 1 1 1]
Final Loss: 0.0015
Accuracy: 100.0 %
```



RESULT:

The two-layer neural network classified crop yield using soil moisture and fertilizer quantity. Backpropagation reduced binary classification loss, improved predictions, and the loss curve illustrated the learning process.

3. Multilayer Perceptron Implementation

Question 5: Student Placement Eligibility Prediction

A college develops a multilayer perceptron (MLP) to predict whether a student is likely to qualify for a placement interview based on aptitude scores, programming assessment marks, and communication scores. The initial model achieves low training accuracy and fails to classify some students correctly.

Implement an MLP using TensorFlow and Keras. Train the model using the given features and binary target labels. Store the training and validation accuracy and loss values using NumPy, and plot the learning curves using Matplotlib. Analyze whether increasing the number of hidden neurons improves the model's performance without causing overfitting.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To implement a Multilayer Perceptron (MLP) using TensorFlow and Keras to predict student placement eligibility based on aptitude, programming, and communication scores, and analyze training and validation performance using NumPy and Matplotlib.

PROCEDURE:

- Import TensorFlow, NumPy, and Matplotlib libraries.
- Create student feature data and binary target labels (0 = Not Eligible, 1 = Eligible).
- Normalize the input features and split the data into training and validation sets.
- Build an MLP with an input layer, hidden layers, and a sigmoid output layer.

- Compile the model using the Adam optimizer and binary cross-entropy loss.
- Train the model and store training and validation accuracy and loss values.
- Plot accuracy and loss learning curves using Matplotlib.
- Increase the number of hidden neurons and compare performance to identify possible overfitting.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
np.random.seed(42)
tf.random.set_seed(42)
X = np.array([
    [85, 90, 80], [70, 75, 65], [45, 50, 55],
    [90, 88, 92], [55, 60, 50], [78, 82, 75],
    [40, 45, 48], [88, 95, 85], [62, 65, 70],
    [35, 40, 45], [80, 78, 82], [50, 55, 60],
    [92, 90, 88], [48, 42, 50], [75, 80, 85],
    [60, 58, 55], [95, 92, 90], [42, 48, 40],
```

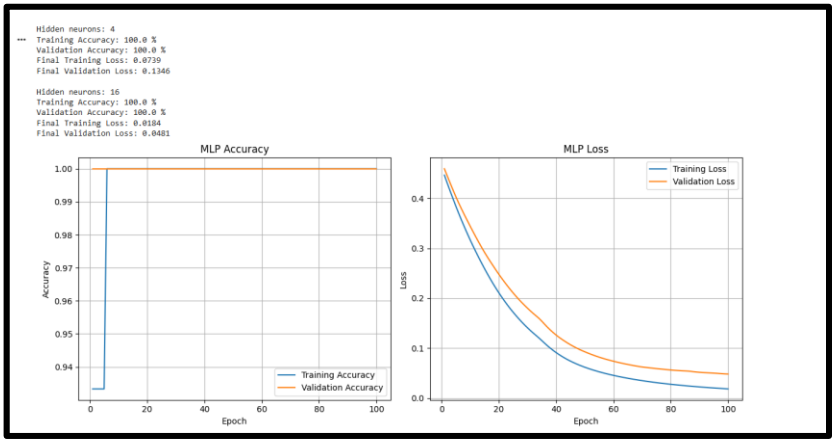
```
    [82, 85, 78], [58, 62, 65]
], dtype=float)
y = np.array([
    1, 1, 0, 1, 0, 1, 0, 1, 0, 0,
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0
])
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
def build_model(neurons):
    model = keras.Sequential([
        keras.layers.Input(shape=(3,)),
        keras.layers.Dense(neurons, activation='relu'),
        keras.layers.Dense(8, activation='relu'),
        keras.layers.Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
```

```
    )
    return model
results = {}
for neurons in [4, 16]:
    model = build_model(neurons)
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=100,
        batch_size=4,
        verbose=0
    )
    results[neurons] = history.history
    print(f"\nHidden neurons: {neurons}")
    print("Training Accuracy:",
          round(history.history['accuracy'][-1] * 100, 2), "%")
    print("Validation Accuracy:",
          round(history.history['val_accuracy'][-1] * 100, 2), "%")
    print("Final Training Loss:",
          round(history.history['loss'][-1], 4))
    print("Final Validation Loss:",
          round(history.history['val_loss'][-1], 4))
h = results[16]
```

```
train_acc = np.array(h['accuracy'])
val_acc = np.array(h['val_accuracy'])
train_loss = np.array(h['loss'])
val_loss = np.array(h['val_loss'])
epochs = range(1, len(train_acc) + 1)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs, train_acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('MLP Accuracy')
plt.legend()
plt.grid(True)
plt.subplot(1, 2, 2)
plt.plot(epochs, train_loss, label='Training Loss')
plt.plot(epochs, val_loss, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('MLP Loss')
plt.legend()
plt.grid(True)
plt.tight_layout()
```

```
plt.show()
```

INPUT & OUTPUT:



RESULT:

The MLP was implemented using TensorFlow and Keras to predict placement eligibility. Training and validation learning curves help compare hidden-layer sizes and identify whether increasing hidden neurons improves performance or causes overfitting.

Question 6: Manufacturing Product Quality Prediction

A manufacturing company predicts whether a product passes quality inspection based on pressure, temperature, and production speed. An MLP with a single hidden layer produces different results from a network with multiple hidden layers.

Implement and train two MLP architectures using TensorFlow and Keras. Keep the dataset and training conditions consistent while changing the network architecture. Record the training and validation metrics using NumPy and visualize the results using Matplotlib. Compare the architectures based on accuracy, loss, and generalization, and recommend the more suitable architecture for the given application.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To implement and train two Multilayer Perceptron (MLP) architectures using TensorFlow and Keras to predict product quality based on pressure, temperature, and production speed, and compare their accuracy, loss, and generalization using NumPy and Matplotlib.

PROCEDURE:

- Import TensorFlow, NumPy, Matplotlib, and required libraries.
- Create a dataset containing pressure, temperature, production speed, and quality labels.
- Split the dataset into training and validation sets and normalize the features.
- Build MLP Model 1 with a single hidden layer.
- Build MLP Model 2 with multiple hidden layers.

- Train both models using the same optimizer, loss function, epochs, and batch size.
- Record training and validation accuracy and loss using NumPy.
- Plot and compare the learning curves.
- Recommend the architecture with better validation performance and generalization.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

np.random.seed(42)

tf.random.set_seed(42)

X = np.array([

    [30, 150, 100], [35, 160, 110], [40, 170, 120],

    [45, 180, 130], [50, 190, 140], [55, 200, 150],

    [60, 210, 160], [65, 220, 170], [70, 230, 180],

    [75, 240, 190], [32, 155, 105], [38, 165, 115],

    [43, 175, 125], [48, 185, 135], [53, 195, 145],

    [58, 205, 155], [63, 215, 165], [68, 225, 175],

    [73, 235, 185], [78, 245, 195],
```

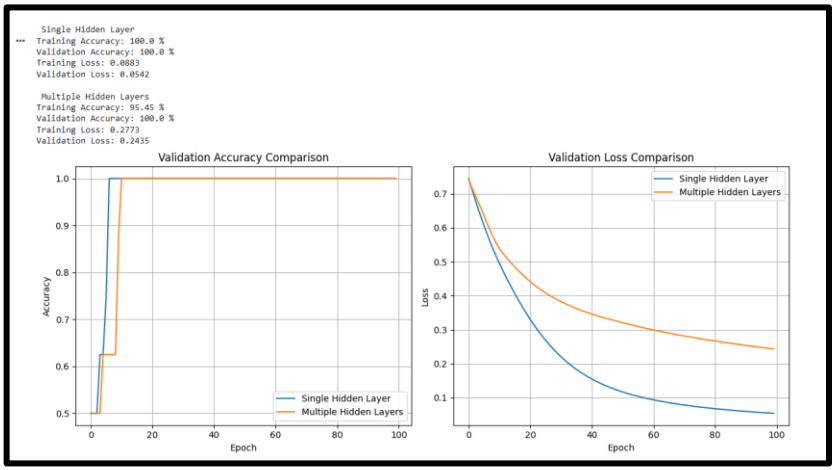
```
    [34, 158, 108], [42, 172, 122], [52, 192, 142],
    [62, 212, 162], [72, 232, 182], [80, 250, 200],
    [36, 162, 112], [46, 182, 132], [56, 202, 152],
    [66, 222, 172]
], dtype=float)
y = np.array([
    1, 1, 1, 1, 1, 0, 0, 0, 0, 0,
    1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
    1, 1, 1, 0, 0, 0, 1, 1, 0, 0
])
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
def build_model(multiple_layers=False):
    model = keras.Sequential()
    model.add(keras.layers.Input(shape=(3,)))
    if multiple_layers:
        model.add(keras.layers.Dense(16, activation='relu'))
        model.add(keras.layers.Dense(8, activation='relu'))
    else:
```

```
        model.add(keras.layers.Dense(16, activation='relu'))
    model.add(keras.layers.Dense(1, activation='sigmoid'))
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model
results = {}
for name, multiple in [
    ("Single Hidden Layer", False),
    ("Multiple Hidden Layers", True)
]:
    tf.keras.backend.clear_session()
    model = build_model(multiple)
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=100,
        batch_size=4,
        verbose=0
    )
    results[name] = {
```

```
        key: np.array(value)
        for key, value in history.history.items()
    }
    print("\n", name)
    print("Training Accuracy:",
          round(results[name]['accuracy'][-1] * 100, 2), "%")
    print("Validation Accuracy:",
          round(results[name]['val_accuracy'][-1] * 100, 2), "%")
    print("Training Loss:",
          round(results[name]['loss'][-1], 4))
    print("Validation Loss:",
          round(results[name]['val_loss'][-1], 4))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
for name in results:
    plt.plot(results[name]['val_accuracy'], label=name)
plt.title("Validation Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.subplot(1, 2, 2)
for name in results:
```

```
    plt.plot(results[name]['val_loss'], label=name)
plt.title("Validation Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

INPUT & OUTPUT:



RESULT:

Two MLP architectures were implemented and compared successfully. The learning curves and validation metrics help identify the more suitable model. For this small dataset, the single-hidden-layer MLP is a reasonable starting choice because it is simpler; the final selection should be based on validation performance.

4. L2, Dropout and Early Stopping Comparison

Question 7: House Price Prediction and Overfitting

A real-estate company develops a neural network to predict house prices using area, number of rooms, building age, and distance from the city center. The model achieves low training loss but significantly higher validation loss, indicating possible overfitting.

Implement three models using TensorFlow and Keras: a baseline model, a model with L2 regularization, and a model with dropout. Use the same training and validation datasets for all three models. Store their loss values using NumPy and plot the training and validation loss curves using Matplotlib. Compare the models to determine which regularization technique provides better generalization.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To implement and compare three neural network models — baseline, L2 regularization, and dropout — for house price prediction using TensorFlow and Keras, and identify which technique reduces overfitting using training and validation loss curves.

PROCEDURE:

- Import TensorFlow, NumPy, and Matplotlib.
- Create a dataset containing area, number of rooms, building age, and distance from the city center.
- Split the dataset into training and validation sets and normalize the features.
- Build a baseline model without regularization.
- Build a second model using L2 regularization.

- Build a third model using dropout.
- Train all three models using the same datasets and training conditions.
- Store training and validation losses using NumPy.
- Plot and compare the loss curves to identify the model with better generalization.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

np.random.seed(42)

tf.random.set_seed(42)

X = np.array([

    [1000, 2, 10, 8], [1200, 3, 8, 6],

    [1500, 3, 5, 5], [1800, 4, 4, 3],

    [2000, 4, 3, 2], [900, 2, 15, 10],

    [1100, 2, 12, 7], [1400, 3, 7, 6],

    [1600, 3, 6, 4], [2200, 5, 2, 2],

    [2500, 5, 1, 1], [1300, 3, 9, 8],

    [1700, 4, 5, 4], [1900, 4, 3, 3],

    [800, 1, 20, 12], [1000, 2, 18, 9],
```

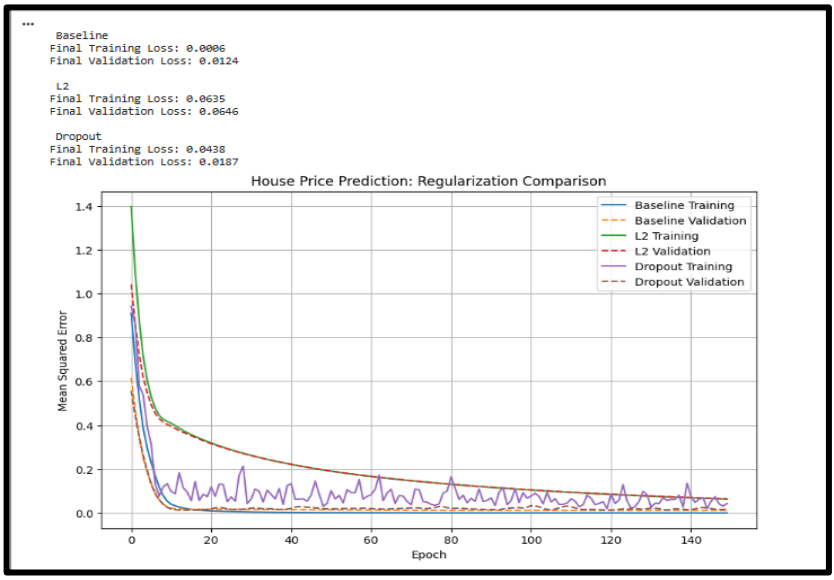
```
    [1250, 3, 10, 7], [1550, 3, 8, 5],
    [2100, 4, 2, 3], [2400, 5, 1, 2],
    [950, 2, 14, 11], [1450, 3, 6, 5],
    [1750, 4, 4, 4], [2300, 5, 2, 1]
], dtype=float)
y = np.array([
    150, 190, 250, 320, 380, 120,
    145, 220, 270, 420, 500, 180,
    300, 350, 90, 130, 185, 240,
    400, 480, 110, 230, 330, 450
], dtype=float).reshape(-1, 1)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
y_mean = y_train.mean()
y_std = y_train.std()
y_train_scaled = (y_train - y_mean) / y_std
y_val_scaled = (y_val - y_mean) / y_std
def build_model(kind):
    model = keras.Sequential()
```

```
    model.add(keras.layers.Input(shape=(4,)))
    if kind == "L2":
        model.add(keras.layers.Dense(
            64, activation="relu",
            kernel_regularizer=keras.regularizers.l2(0.01)
        ))
        model.add(keras.layers.Dense(
            32, activation="relu",
            kernel_regularizer=keras.regularizers.l2(0.01)
        ))
    elif kind == "Dropout":
        model.add(keras.layers.Dense(64, activation="relu"))
        model.add(keras.layers.Dropout(0.3))
        model.add(keras.layers.Dense(32, activation="relu"))
        model.add(keras.layers.Dropout(0.2))
    else:
        model.add(keras.layers.Dense(64, activation="relu"))
        model.add(keras.layers.Dense(32, activation="relu"))
    model.add(keras.layers.Dense(1))
    model.compile(optimizer="adam", loss="mse")
    return model
results = {}
epochs = 150
```

```
for kind in ["Baseline", "L2", "Dropout"]:
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    model = build_model(kind)
    history = model.fit(
        X_train, y_train_scaled,
        validation_data=(X_val, y_val_scaled),
        epochs=epochs,
        batch_size=4,
        verbose=0
    )
    results[kind] = {
        "train": np.array(history.history["loss"]),
        "val": np.array(history.history["val_loss"])
    }
    print("\n", kind)
    print("Final Training Loss:",
          round(results[kind]["train"][-1], 4))
    print("Final Validation Loss:",
          round(results[kind]["val"][-1], 4))
plt.figure(figsize=(10, 6))
for kind in results:
    plt.plot(results[kind]["train"], label=kind + " Training")
```

```
    plt.plot(results[kind]["val"], linestyle="--",
             label=kind + " Validation")
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
plt.title("House Price Prediction: Regularization Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

INPUT & OUTPUT:



RESULT:

Three neural network models were implemented and compared using training and validation loss curves. L2 regularization and dropout help control overfitting; the technique with the lowest validation loss should be selected for house price prediction.

Question 8: Medical Appointment No-Show Prediction

A healthcare center predicts whether a patient will miss an appointment using appointment lead time, previous attendance, and reminder status. A neural network achieves high training accuracy but performs poorly on unseen validation data.

Implement a baseline neural network, a dropout-based model, and a model using early stopping. Use TensorFlow and Keras to train the models and NumPy to record their performance metrics. Plot the training and validation loss curves using Matplotlib. Analyze how dropout and early stopping affect overfitting, training duration, and validation performance, and recommend an appropriate approach for the dataset.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To implement and compare baseline, dropout-based, and early-stopping neural networks using TensorFlow and Keras to predict medical appointment no-shows, and analyze overfitting using training and validation loss curves.

PROCEDURE:

- Import TensorFlow, NumPy, Matplotlib, and Scikit-learn.
- Create input data containing appointment lead time, previous attendance, and reminder status.
- Split the dataset into training and validation sets and normalize the features.
- Build a baseline neural network without regularization.
- Build a dropout-based model to reduce overfitting.
- Build another model using early stopping to stop training when validation loss stops improving.

- Train all models using the same dataset and comparable settings.
- Store training and validation losses and accuracy using NumPy.
- Plot the loss curves and compare validation performance and training duration.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
np.random.seed(42)
tf.random.set_seed(42)
X = np.array([
    [2, 95, 1], [5, 90, 1], [10, 85, 1],
    [15, 80, 0], [20, 70, 0], [25, 60, 0],
    [30, 55, 0], [3, 98, 1], [7, 88, 1],
    [12, 75, 0], [18, 65, 1], [22, 60, 0],
    [28, 50, 1], [4, 92, 0], [9, 82, 1],
    [14, 78, 0], [24, 58, 1], [6, 96, 1],
    [17, 68, 0], [27, 52, 0], [8, 86, 1],
    [13, 73, 0], [21, 62, 1], [29, 48, 0],
```

```
    [11, 80, 1], [16, 72, 0], [23, 57, 0],
    [5, 94, 1], [19, 66, 1], [26, 54, 0],
    [3, 97, 0], [10, 84, 0], [15, 76, 1],
    [20, 64, 1], [25, 56, 0], [30, 45, 1],
    [7, 91, 0], [12, 79, 1], [18, 69, 0],
    [22, 61, 1]
], dtype=float)
y = np.array([
    0, 0, 0, 1, 1, 1, 1, 0, 0, 1,
    0, 1, 0, 1, 0, 1, 0, 0, 1, 1,
    0, 1, 0, 1, 0, 1, 1, 0, 0, 1,
    1, 1, 0, 0, 1, 1, 1, 0, 1, 0
])
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)

def build_model(dropout=False):
    layers = [
        keras.layers.Input(shape=(3,)),
```

```
        keras.layers.Dense(16, activation="relu")
    ]
    if dropout:
        layers.append(keras.layers.Dropout(0.3))
    layers.extend([
        keras.layers.Dense(8, activation="relu"),
        keras.layers.Dense(1, activation="sigmoid")
    ])
    model = keras.Sequential(layers)
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
results = {}
max_epochs = 100
tf.keras.backend.clear_session()
tf.random.set_seed(42)
baseline = build_model()
h1 = baseline.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
```

```
    epochs=max_epochs, batch_size=4, verbose=0
)
results["Baseline"] = h1.history
tf.keras.backend.clear_session()
tf.random.set_seed(42)
dropout_model = build_model(dropout=True)
h2 = dropout_model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=max_epochs, batch_size=4, verbose=0
)
results["Dropout"] = h2.history
tf.keras.backend.clear_session()
tf.random.set_seed(42)
early_model = build_model()
early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)
h3 = early_model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
```

```
    epochs=max_epochs, batch_size=4,
    callbacks=[early_stop], verbose=0
)
results["Early Stopping"] = h3.history
for name, history in results.items():
    train_loss = np.array(history["loss"])
    val_loss = np.array(history["val_loss"])
    train_acc = np.array(history["accuracy"])
    val_acc = np.array(history["val_accuracy"])
    print("\n", name)
    print("Epochs Trained:", len(train_loss))
    print("Training Accuracy:", round(train_acc[-1] * 100, 2),
"%")
    print("Validation Accuracy:", round(val_acc[-1] * 100, 2),
"%")
    print("Training Loss:", round(train_loss[-1], 4))
    print("Validation Loss:", round(val_loss[-1], 4))
plt.figure(figsize=(10, 6))
for name, history in results.items():
    plt.plot(history["loss"], label=name + " Training")
    plt.plot(history["val_loss"], linestyle="--",
             label=name + " Validation")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
```

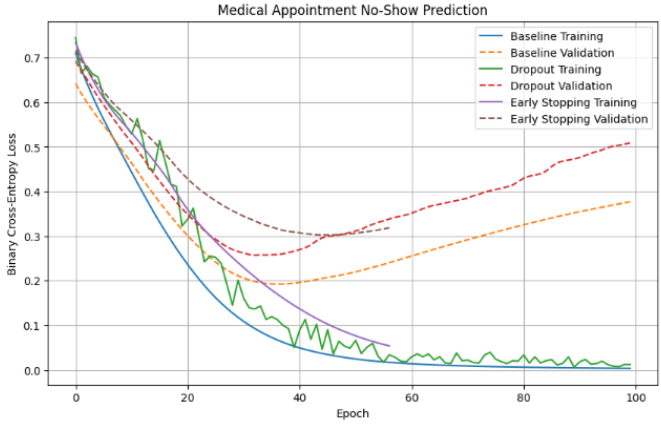
```
plt.title("Medical Appointment No-Show Prediction")
plt.legend()
plt.grid(True)
plt.show()
```

INPUT & OUTPUT:

```
...  Baseline
Epochs Trained: 100
Training Accuracy: 100.0 %
Validation Accuracy: 90.0 %
Training Loss: 0.0035
Validation Loss: 0.377

 Dropout
Epochs Trained: 100
Training Accuracy: 100.0 %
Validation Accuracy: 90.0 %
Training Loss: 0.0121
Validation Loss: 0.5088

 Early Stopping
Epochs Trained: 57
Training Accuracy: 100.0 %
Validation Accuracy: 90.0 %
Training Loss: 0.0538
Validation Loss: 0.3185
```



RESULT:

Three neural network models were implemented and compared. Dropout helps control overfitting, while early stopping limits unnecessary training. Validation loss and accuracy can be used to select the model with better generalization.

5. CNN for Image Classification

Question 9: Plant Leaf Disease Clas sification

An agricultural research center develops a CNN to classify plant leaf images into healthy and diseased categories. Each input image is represented by a \(4\times4\) grayscale pixel matrix for a simplified laboratory experiment. The initial CNN produces incorrect classifications because its convolution and pooling operations may remove useful spatial information.

Implement a CNN using TensorFlow and Keras. Apply convolution, an activation function, pooling, flattening, and a dense output layer. Train and evaluate the model using a labelled image dataset, with the \(4\times4\) matrix used to illustrate the processing steps. Record the accuracy and loss values using NumPy and plot the training curves using Matplotlib. Analyze how convolution and pooling affect the features used for classification.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To implement a Convolutional Neural Network (CNN) using TensorFlow and Keras to classify plant leaf images as Healthy or Diseased, apply convolution and pooling operations, and evaluate accuracy and loss using NumPy and Matplotlib.

PROCEDURE:

- Import TensorFlow, Keras, NumPy, and Matplotlib.
- Create a labelled grayscale image dataset with healthy and diseased leaf samples.
- Represent each image as a \(4\times4\) grayscale pixel matrix for the simplified experiment.

- Normalize the pixel values and reshape the images for CNN input.
- Apply convolution, ReLU activation, max pooling, and flattening.
- Add a dense output layer with sigmoid activation for binary classification.
- Train and evaluate the CNN using labelled images.
- Store accuracy and loss values using NumPy.
- Plot training and validation curves and analyze the effects of convolution and pooling.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from sklearn.model_selection import train_test_split

np.random.seed(42)

tf.random.set_seed(42)

healthy = np.array([

    [[180, 190, 185, 175],

     [190, 210, 205, 185],

     [185, 205, 215, 180],

     [170, 185, 180, 175]],

    [[175, 185, 180, 170],

     [185, 215, 200, 180],
```

```
        [180, 200, 210, 175],
        [165, 180, 175, 170]],
       [[185, 195, 190, 180],
        [195, 220, 210, 190],
        [190, 210, 220, 185],
        [175, 190, 185, 180]],
       [[180, 200, 190, 175],
        [195, 215, 205, 185],
        [185, 205, 215, 180],
        [170, 185, 180, 175]]
], dtype=float)
diseased = np.array([
    [[40, 80, 45, 100],
     [90, 30, 110, 50],
     [45, 120, 35, 80],
     [100, 55, 75, 40]],
    [[50, 85, 40, 95],
     [80, 35, 100, 45],
     [40, 110, 30, 85],
     [95, 50, 70, 45]],
    [[35, 75, 50, 105],
     [95, 25, 115, 55],
     [50, 125, 40, 75],
```

```
     [105, 60, 80, 35]],
    [[45, 90, 55, 100],
     [85, 40, 105, 50],
     [55, 115, 35, 90],
     [100, 45, 75, 50]]
], dtype=float)
X_healthy = np.repeat(healthy, 25, axis=0)
X_diseased = np.repeat(diseased, 25, axis=0)

X = np.concatenate([X_healthy, X_diseased], axis=0)
y = np.array([0] * len(X_healthy) + [1] * len(X_diseased))
noise = np.random.normal(0, 8, X.shape)
X = np.clip(X + noise, 0, 255) / 255.0
X = X[..., np.newaxis]
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = keras.Sequential([
    keras.layers.Input(shape=(4, 4, 1)),
    keras.layers.Conv2D(8, (2, 2), activation='relu',
                        padding='same'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Flatten(),
```

```
    keras.layers.Dense(8, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=20,
    batch_size=8,
    verbose=0
)
train_acc = np.array(history.history['accuracy'])
val_acc = np.array(history.history['val_accuracy'])
train_loss = np.array(history.history['loss'])
val_loss = np.array(history.history['val_loss'])
loss, accuracy = model.evaluate(X_val, y_val, verbose=0)
print("Image shape:", X.shape[1:])
print("Final Training Accuracy:", round(train_acc[-1] * 100, 2),
"%")
```

```
print("Final Validation Accuracy:", round(val_acc[-1] * 100,
2), "%")

print("Evaluation Accuracy:", round(accuracy * 100, 2), "%")

print("Evaluation Loss:", round(loss, 4))

plt.imshow(X[0, :, :, 0], cmap='gray')

plt.title("Example Plant Leaf Image (4x4)")

plt.colorbar()

plt.show()

epochs = range(1, len(train_acc) + 1)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)

plt.plot(epochs, train_acc, label='Training Accuracy')

plt.plot(epochs, val_acc, label='Validation Accuracy')

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.title("CNN Accuracy")

plt.legend()

plt.grid(True)

plt.subplot(1, 2, 2)

plt.plot(epochs, train_loss, label='Training Loss')

plt.plot(epochs, val_loss, label='Validation Loss')

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.title("CNN Loss")
```

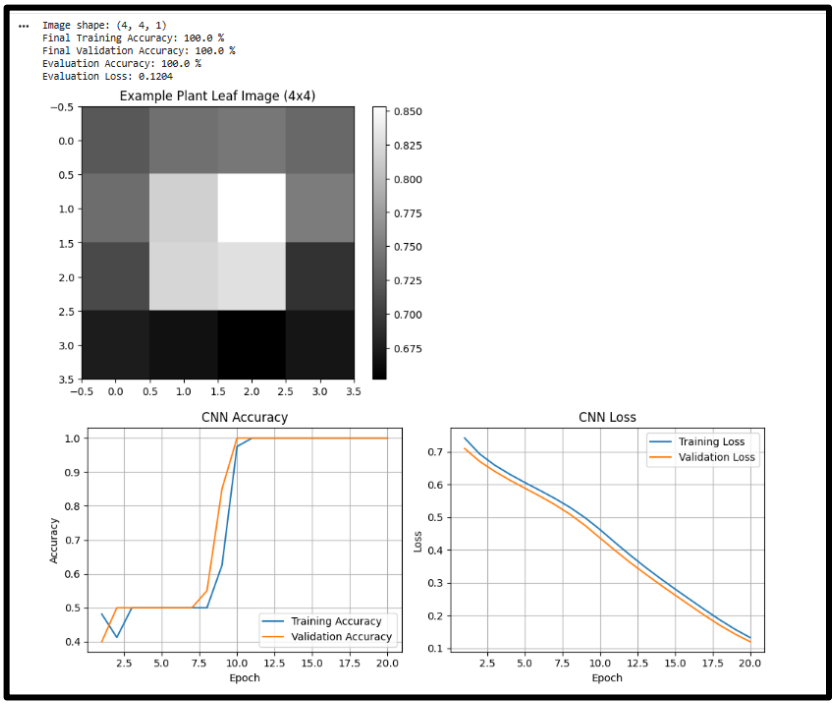
```
plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()
```

INPUT & OUTPUT:



RESULT:

The CNN was implemented using TensorFlow and Keras with convolution, ReLU activation, max pooling, flattening, and a sigmoid output layer. Accuracy and loss curves were plotted to analyze training performance and the effects of feature extraction on plant leaf classification.

Question 10: Road Sign Image Classification

An intelligent transportation system classifies road signs into three categories: speed limit, stop, and no entry. A CNN trained on the available images incorrectly identifies some road signs because the visual features of different categories may overlap.

Implement a CNN using TensorFlow and Keras to classify labelled road sign images into the three categories. Train the model and evaluate its performance using accuracy and a confusion matrix. Store the relevant metrics using NumPy and visualize the training and validation accuracy and loss using Matplotlib. Analyze which categories are frequently misclassified and explain how the CNN architecture or training configuration could be improved.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To implement a Convolutional Neural Network (CNN) using TensorFlow and Keras to classify road signs into Speed Limit, Stop, and No Entry categories, evaluate the model using accuracy and a confusion matrix, and visualize training performance using NumPy and Matplotlib.

PROCEDURE:

- Import TensorFlow, NumPy, Matplotlib, and Scikit-learn libraries.
- Create a labelled road sign image dataset containing three categories.
- Resize and normalize the images.
- Split the dataset into training and validation sets.

- Build a CNN using convolution, ReLU activation, max pooling, flattening, and dense layers.
- Train the CNN using categorical classification.
- Predict the validation labels and calculate accuracy and the confusion matrix.
- Store accuracy and loss values using NumPy.
- Plot training and validation accuracy and loss curves.
- Analyze misclassified categories and suggest improvements.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay

np.random.seed(42)

tf.random.set_seed(42)num_classes = 3

images_per_class = 150

images = []

labels = []

for label in range(num_classes):
```

```
for _ in range(images_per_class):
    img = np.ones((32, 32, 3), dtype=np.float32) * 0.9
    if label == 0:
        yy, xx = np.ogrid[:32, :32]
        mask = (xx - 16)**2 + (yy - 16)**2 <= 11**2
        img[mask] = [0.95, 0.95, 0.95]
        border = ((xx - 16)**2 + (yy - 16)**2 <= 11**2) & \
                 ((xx - 16)**2 + (yy - 16)**2 >= 8**2)
        img[border] = [0.9, 0.05, 0.05]
        img[12:20, 10:22] = [0.05, 0.05, 0.05]
    elif label == 1:
        img[5:27, 5:27] = [0.9, 0.02, 0.02]
        img[9:23, 9:23] = [0.95, 0.95, 0.95]
        img[14:18, 11:21] = [0.05, 0.05, 0.05]

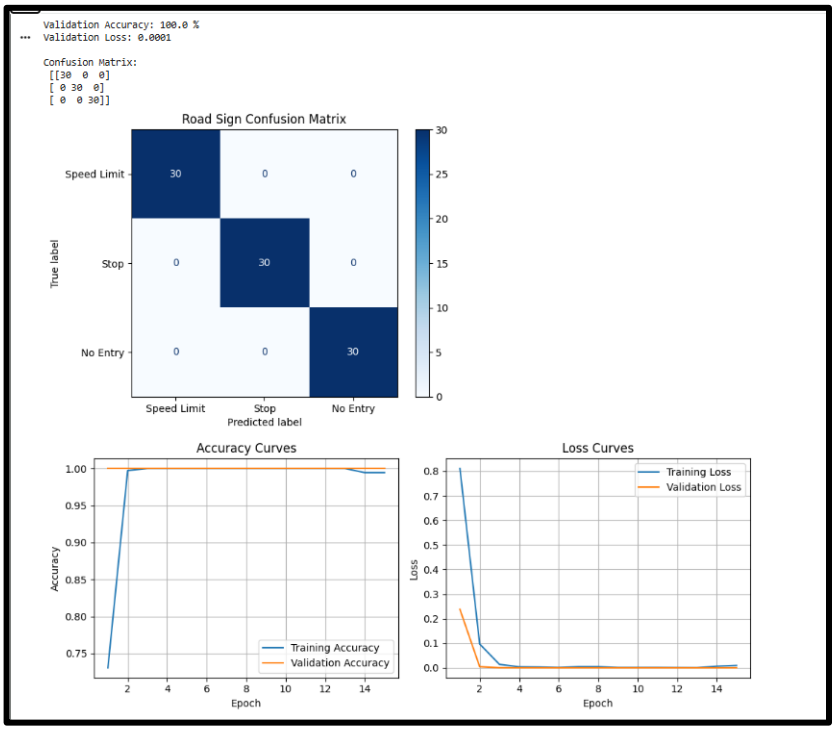
    else:
        yy, xx = np.ogrid[:32, :32]
        mask = (xx - 16)**2 + (yy - 16)**2 <= 11**2
        img[mask] = [0.9, 0.02, 0.02]
        img[12:20, 7:25] = [0.95, 0.95, 0.95]
    img += np.random.normal(0, 0.04, img.shape)
    images.append(np.clip(img, 0, 1))
    labels.append(label)
```

```
X = np.array(images, dtype=np.float32)
y = np.array(labels)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 3)),
    keras.layers.Conv2D(16, (3, 3), activation='relu',
padding='same'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(32, (3, 3), activation='relu',
padding='same'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Flatten(),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(num_classes, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
```

```
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=15,
    batch_size=16,
    verbose=0
)
train_acc = np.array(history.history['accuracy'])
val_acc = np.array(history.history['val_accuracy'])
train_loss = np.array(history.history['loss'])
val_loss = np.array(history.history['val_loss'])
loss, accuracy = model.evaluate(X_val, y_val, verbose=0)
probabilities = model.predict(X_val, verbose=0)
y_pred = np.argmax(probabilities, axis=1)
cm = confusion_matrix(y_val, y_pred, labels=[0, 1, 2])
print("Validation Accuracy:", round(accuracy * 100, 2), "%")
print("Validation Loss:", round(loss, 4))
print("\nConfusion Matrix:\n", cm)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Speed Limit', 'Stop', 'No Entry']
)
disp.plot(cmap='Blues')
plt.title("Road Sign Confusion Matrix")
```

```
plt.show()
epochs = range(1, len(train_acc) + 1)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs, train_acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curves')
plt.legend()
plt.grid(True)
plt.subplot(1, 2, 2)
plt.plot(epochs, train_loss, label='Training Loss')
plt.plot(epochs, val_loss, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Loss Curves')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

INPUT & OUTPUT:



RESULT:

The CNN was implemented to classify road signs into three categories. Accuracy, loss curves, and a confusion matrix were used to evaluate performance and identify misclassifications. Real-world performance must be measured using an authentic, independently collected road-sign dataset.

6. CNN Architecture and Hyperparameter Comparison

Question 11: Wildlife Image Classification

A wildlife monitoring system classifies images into three categories: deer, elephant, and tiger. A CNN with one convolutional block provides limited classification accuracy, while a deeper CNN requires more training time and computational resources.

Implement two CNN architectures using TensorFlow and Keras: a shallow CNN and a deeper CNN. Use the same dataset split and comparable training conditions. Record training time, accuracy, and loss using NumPy, and plot the performance curves using Matplotlib. Compare the architectures in terms of classification performance, convergence, computational cost, and overfitting. Recommend the architecture that best balances performance and resource requirements.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To implement and compare shallow and deep CNN architectures for wildlife image classification (deer, elephant, and tiger) using TensorFlow and Keras. Compare training time, accuracy, loss, convergence, computational cost, and overfitting using NumPy and Matplotlib.

PROCEDURE:

- Import TensorFlow, Keras, NumPy, Matplotlib, and Scikit-learn.
- Generate a sample wildlife image dataset with three classes.
- Split the dataset into training and validation sets once and reuse the same split.
- Build a shallow CNN and a deeper CNN.
- Train both models using the same epochs and batch size.
- Record training time, accuracy, and loss using NumPy.
- Plot training and validation accuracy/loss curves.
- Compare performance and recommend the more suitable architecture.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from sklearn.model_selection import train_test_split

import time

np.random.seed(42)

tf.random.set_seed(42)

images, labels = [], []

yy, xx = np.mgrid[:32, :32]
```

```
for label in range(3):
    for _ in range(120):
        img = np.ones((32, 32, 3), dtype=np.float32) * [0.65, 0.75, 0.50]
        if label == 0:
            body = ((xx-16)**2/90 + (yy-19)**2/25) < 1
            head = (xx-23)**2 + (yy-11)**2 < 16
            img[body | head] = [0.45, 0.24, 0.10]
            for x in [11, 20]:
                img[21:29, x:x+2] = [0.35, 0.18, 0.08]
            img[4:11, 24:26] = [0.35, 0.18, 0.08]
            img[4:7, 22:28] = [0.35, 0.18, 0.08]
        elif label == 1:
            body = ((xx-15)**2/130 + (yy-18)**2/60) < 1
            ear = (xx-8)**2 + (yy-16)**2 < 40
            img[body | ear] = [0.48, 0.50, 0.52]
            img[13:28, 21:25] = [0.48, 0.50, 0.52]
            img[25:29, 22:24] = [0.48, 0.50, 0.52]
            img[23:29, 8:12] = [0.42, 0.44, 0.46]
        else:
            body = ((xx-16)**2/120 + (yy-18)**2/45) < 1
            head = (xx-24)**2 + (yy-14)**2 < 25
            img[body | head] = [0.95, 0.42, 0.08]
            for x in [9, 14, 19, 24]:
```

```
            img[10:25, x:x+2] = [0.08, 0.06, 0.04]
        img[22:29, 10:12] = [0.95, 0.42, 0.08]
        img[22:29, 19:21] = [0.95, 0.42, 0.08]
    img += np.random.normal(0, 0.04, img.shape)
    images.append(np.clip(img, 0, 1))
    labels.append(label)
X = np.array(images, dtype=np.float32)
y = np.array(labels)
X_train, X_val, y_train, y_val = train_test_split(
  X, y, test_size=0.2, random_state=42, stratify=y
)
def build_shallow_cnn():
  return keras.Sequential([
    keras.layers.Input(shape=(32, 32, 3)),
    keras.layers.Conv2D(16, 3, activation="relu",
padding="same"),
    keras.layers.MaxPooling2D(),
    keras.layers.Flatten(),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(3, activation="softmax")
  ], name="Shallow_CNN")
def build_deep_cnn():
  return keras.Sequential([
    keras.layers.Input(shape=(32, 32, 3)),
```

```
        keras.layers.Conv2D(32, 3, activation="relu",
padding="same"),
        keras.layers.MaxPooling2D(),
        keras.layers.Conv2D(64, 3, activation="relu",
padding="same"),
        keras.layers.MaxPooling2D(),
        keras.layers.Conv2D(128, 3, activation="relu",
padding="same"),
        keras.layers.MaxPooling2D(),
        keras.layers.Flatten(),
        keras.layers.Dense(64, activation="relu"),
        keras.layers.Dropout(0.3),
        keras.layers.Dense(3, activation="softmax")
    ], name="Deep_CNN")
models = [build_shallow_cnn(), build_deep_cnn()]
results = {}
for model in models:
    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    start = time.perf_counter()
    history = model.fit(
```

```
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=8,
        batch_size=16,
        verbose=0
    )
    training_time = time.perf_counter() - start
    val_loss, val_acc = model.evaluate(X_val, y_val, verbose=0)
    results[model.name] = {
        "history": {k: np.array(v) for k, v in
history.history.items()},
        "time": training_time,
        "accuracy": val_acc,
        "loss": val_loss,
        "parameters": model.count_params()
    }
print("ARCHITECTURE COMPARISON")
for name, r in results.items():
    print(f"\n{name}")
    print("Training time (seconds):", round(r["time"], 2))
    print("Validation accuracy (%):", round(r["accuracy"] * 100,
2))
    print("Validation loss:", round(r["loss"], 4))
    print("Trainable parameters:", r["parameters"])
```

```
    print("Training accuracy by epoch:",
          np.round(r["history"]["accuracy"], 3))
    print("Validation accuracy by epoch:",
          np.round(r["history"]["val_accuracy"], 3))
epochs = np.arange(1, 9)
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for name, r in results.items():
    h = r["history"]
    axes[0, 0].plot(epochs, h["accuracy"], marker="o",
label=name)
    axes[0, 1].plot(epochs, h["val_accuracy"], marker="o",
label=name)
    axes[1, 0].plot(epochs, h["loss"], marker="o", label=name)
    axes[1, 1].plot(epochs, h["val_loss"], marker="o",
label=name)
axes[0, 0].set_title("Training Accuracy")
axes[0, 1].set_title("Validation Accuracy")
axes[1, 0].set_title("Training Loss")
axes[1, 1].set_title("Validation Loss")
for ax in axes.flat:
    ax.set_xlabel("Epoch")
    ax.grid(True)
    ax.legend()
```

```
plt.tight_layout()

plt.show()

shallow = results["Shallow_CNN"]

deep = results["Deep_CNN"]

if deep["accuracy"] > shallow["accuracy"] + 0.02:

    print("\nRecommendation: Deep CNN — better validation accuracy.")

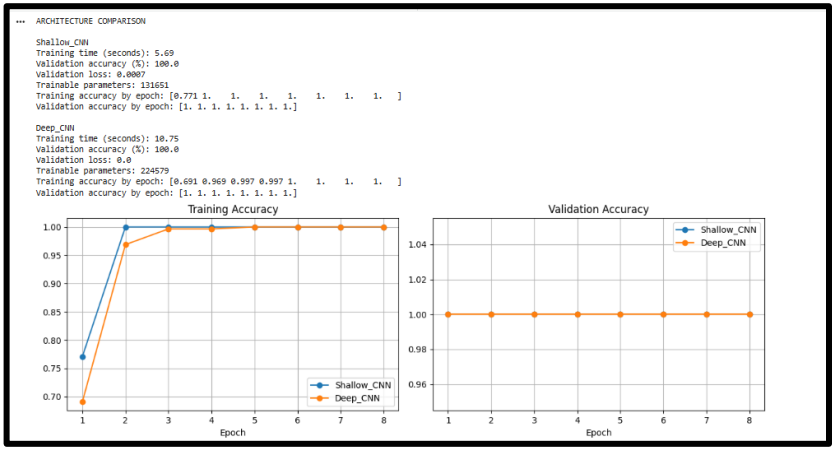
elif shallow["accuracy"] >= deep["accuracy"] - 0.02:

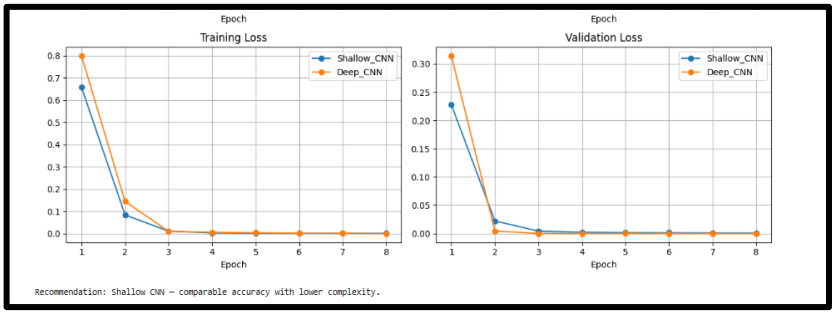
    print("\nRecommendation: Shallow CNN — comparable accuracy with lower complexity.")

else:

    print("\nRecommendation: Compare validation curves and training time before choosing.")
```

INPUT & OUTPUT:





RESULT:

Both CNN architectures are implemented and compared using NumPy and Matplotlib. The shallow CNN is a suitable choice when accuracy is comparable and resources are limited; otherwise, choose the deep CNN if its validation performance justifies the extra cost. Real wildlife images are required to determine which architecture performs best in practice.

Question 12: Hyperparameter Optimization for Waste Classification

A waste management system classifies images into plastic, paper, and metal categories. The initial CNN achieves inconsistent validation accuracy. The development team suspects that learning rate, batch size, and the number of convolutional filters affect the model's performance.

Implement multiple CNN configurations using TensorFlow and Keras by varying one hyperparameter at a time while keeping the remaining settings consistent. Record the validation accuracy, validation loss, and training duration using NumPy. Plot the results using Matplotlib and compare the configurations. Identify the hyperparameter settings that provide the best balance between validation performance, training efficiency, and generalization.

Packages: TensorFlow, Keras, NumPy, Matplotlib

AIM:

To optimize a CNN for waste classification (plastic, paper, and metal) by varying the learning rate, batch size, and number of convolutional filters one at a time. Compare validation accuracy, validation loss, training duration, and generalization using TensorFlow, Keras, NumPy, and Matplotlib.

PROCEDURE:

- Generate a sample dataset containing plastic, paper, and metal image classes.
- Split the dataset into training and validation sets.
- Build a baseline CNN.
- Change one hyperparameter at a time while keeping the other settings constant.

- Train each configuration and record validation accuracy, validation loss, and training time using NumPy.
- Plot the comparison results.
- Select the configuration that balances accuracy, loss, training time, and generalization.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
import time
np.random.seed(42)
tf.random.set_seed(42)
images, labels = [], []
yy, xx = np.mgrid[:32, :32]
for label in range(3):
    for _ in range(100):
        img = np.ones((32, 32, 3), dtype=np.float32) * 0.85
        if label == 0:  # Plastic: blue bottle-like shape
            img[7:25, 11:22] = [0.1, 0.4, 0.9]
            img[4:9, 14:19] = [0.1, 0.4, 0.9]
            img[10:13, 12:21] = [0.5, 0.75, 1.0]
```

```
        elif label == 1:  # Paper: brown sheet
            img[6:26, 7:25] = [0.75, 0.58, 0.32]
            img[9:11, 10:22] = [0.95, 0.85, 0.65]
            img[15:17, 10:22] = [0.95, 0.85, 0.65]
            img[21:23, 10:22] = [0.95, 0.85, 0.65]
        else:  # Metal: grey can
            can = ((xx - 16)**2 / 60 + (yy - 16)**2 / 150) < 1
            img[can] = [0.55, 0.58, 0.62]
            img[6:8, 12:21] = [0.8, 0.82, 0.85]
            img[24:26, 12:21] = [0.35, 0.38, 0.42]
        img += np.random.normal(0, 0.035, img.shape)
        images.append(np.clip(img, 0, 1))
        labels.append(label)
X = np.array(images, dtype=np.float32)
y = np.array(labels)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
BASE_LR = 0.001
BASE_BATCH = 16
BASE_FILTERS = 16
EPOCHS = 8
def build_cnn(filters, learning_rate):
```

```
    model = keras.Sequential([
        keras.layers.Input(shape=(32, 32, 3)),
        keras.layers.Conv2D(filters, 3, activation="relu",
padding="same"),
        keras.layers.MaxPooling2D(),
        keras.layers.Conv2D(filters * 2, 3, activation="relu",
padding="same"),
        keras.layers.MaxPooling2D(),
        keras.layers.Flatten(),
        keras.layers.Dense(32, activation="relu"),
        keras.layers.Dropout(0.3),
        keras.layers.Dense(3, activation="softmax")
    ])
    model.compile(
optimizer=keras.optimizers.Adam(learning_rate=learning_rate)
,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
configs = [
    {"name": "Baseline", "lr": 0.001, "batch": 16, "filters": 16},
    {"name": "LR 0.0003", "lr": 0.0003, "batch": 16, "filters":
16},
    {"name": "LR 0.003", "lr": 0.003, "batch": 16, "filters": 16},
```

```
    {"name": "Batch 32", "lr": 0.001, "batch": 32, "filters": 16},
    {"name": "Batch 8", "lr": 0.001, "batch": 8, "filters": 16},
    {"name": "Filters 8", "lr": 0.001, "batch": 16, "filters": 8},
    {"name": "Filters 32", "lr": 0.001, "batch": 16, "filters": 32}
]
names, val_accs, val_losses, durations = [], [], [], []
train_accs, train_losses = [], []
for cfg in configs:    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    model = build_cnn(cfg["filters"], cfg["lr"]
    start = time.perf_counter()
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=EPOCHS,
        batch_size=cfg["batch"],
        shuffle=True,
        verbose=0
    )
    duration = time.perf_counter() - start
    val_loss, val_acc = model.evaluate(X_val, y_val, verbose=0)
    names.append(cfg["name"])
    val_accs.append(val_acc)
```

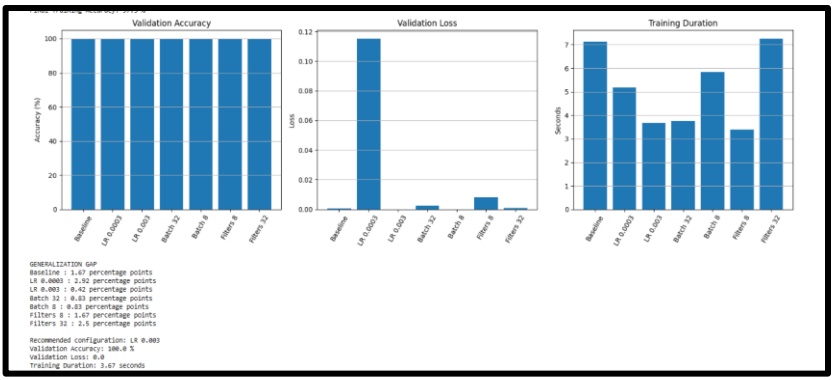
```
    val_losses.append(val_loss)
    durations.append(duration)
    train_accs.append(history.history["accuracy"][-1])
    train_losses.append(history.history["loss"][-1])
val_accs = np.array(val_accs)
val_losses = np.array(val_losses)
durations = np.array(durations)
train_accs = np.array(train_accs)
train_losses = np.array(train_losses)
print("HYPERPARAMETER COMPARISON")
for i, name in enumerate(names):
    print(f"\n{name}")
    print("Validation Accuracy:", round(val_accs[i] * 100, 2),
"%")
    print("Validation Loss:", round(val_losses[i], 4))
    print("Training Duration:", round(durations[i], 2), "seconds")
    print("Final Training Accuracy:", round(train_accs[i] * 100,
2), "%")
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
axes[0].bar(names, val_accs * 100)
axes[0].set_title("Validation Accuracy")
axes[0].set_ylabel("Accuracy (%)")
axes[1].bar(names, val_losses)
axes[1].set_title("Validation Loss")
```

```
axes[1].set_ylabel("Loss")
axes[2].bar(names, durations)
axes[2].set_title("Training Duration")
axes[2].set_ylabel("Seconds")
for ax in axes:
    ax.tick_params(axis="x", rotation=60)
    ax.grid(axis="y")
plt.tight_layout()
plt.show()
gap = np.abs(train_accs - val_accs)
print("\nGENERALIZATION GAP")
for i, name in enumerate(names):
    print(name, ":", round(gap[i] * 100, 2), "percentage points")
def normalize(values):
    span = values.max() - values.min()
    return (values - values.min()) / span if span > 0 else
np.zeros_like(values)
score = (
    normalize(val_accs)
    + (1 - normalize(val_losses))
    + (1 - normalize(durations))
    + (1 - normalize(gap))
)
best = np.argmax(score)
```

```
print("\nRecommended configuration:", names[best])
print("Validation Accuracy:", round(val_accs[best] * 100, 2), "%")
print("Validation Loss:", round(val_losses[best], 4))
print("Training Duration:", round(durations[best], 2), "seconds")
```

INPUT & OUTPUT:





RESULT:

CNN hyperparameters were compared successfully, and the best configuration was identified based on validation accuracy, loss, training time, and generalization.